

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

MLPY: A Modular Machine Learning Framework for Python with Native Spatial Support

Francisco Parra O.
Universidad de Santiago de Chile

Giuseppe Cavallieri
Universidad de Santiago de Chile

Abstract

Python's machine learning ecosystem is dominated by **scikit-learn**, a powerful but monolithic framework where preprocessing, modeling, and evaluation are tightly coupled. This makes it hard to compose complex pipelines from reusable components or extend the framework cleanly. We present **MLPY**, a modular machine learning framework for Python inspired by R's **mlr3**. **MLPY** separates tasks, learners, measures, and predictions into composable abstractions, enabling systematic comparison of methods and clean extension.

The key contribution is native support for spatial machine learning—currently missing in Python. Our Meuse dataset case study shows that regular cross-validation underestimates prediction error by 19% on spatial data ($RMSE$ = 104 ppm vs. 125 ppm with spatial CV). A robustness analysis across multiple k-fold configurations (k=3, 5, 7, 10) confirms this finding is not configuration-dependent, with underestimation ranging from 15.6% to 32.7% (mean: 24.7%). We provide practical guidance on when to use spatial CV based on Moran's I statistic. The framework is available as open-source software on PyPI (`pip install mlpy-geo`).

Keywords: machine learning, spatial statistics, cross-validation, Python, spatial autocorrelation.

1. Introduction

Python's machine learning ecosystem is dominated by **scikit-learn** (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg *et al.* 2011), a powerful but monolithic framework. While **scikit-learn** excels at providing consistent APIs for individual algorithms, it tightly couples preprocessing, modeling, and evaluation. This makes it hard to systematically compare multiple learners across multiple datasets, compose complex pipelines from reusable components, or extend the framework with custom methods.

Want to systematically compare multiple learners across multiple datasets? You'll write a lot

of boilerplate. Need spatial cross-validation to handle geographic data? Not available. Want to tune hyperparameters declaratively without manually calculating grid points? You'll build it yourself.

We present **MLPY**, a modular machine learning framework for Python inspired by R's **mlr3** (Lang, Binder, Richter, Schratz, Pfisterer, Coors, Au, Casalicchio, Kotthoff, and Bischl 2019). **MLPY** separates tasks, learners, measures, and predictions into composable abstractions. This separation enables:

- **Systematic benchmarking**: Compare multiple learners across multiple datasets with a single function call.
- **Reusable pipelines**: Build complex preprocessing graphs that work with any learner.
- **Spatial cross-validation**: Native support for geographic data that respects spatial structure.
- **Declarative tuning**: Specify hyperparameter spaces symbolically rather than manually.
- **Clean extension**: Add new learners, measures, or resampling strategies by inheriting from base classes.

The key contribution is native support for spatial machine learning. When machine learning is applied to spatial or geographic data, standard cross-validation can produce overly optimistic performance measures because spatially proximal observations tend to be more similar than distant ones (spatial autocorrelation). **MLPY** provides spatial resampling strategies that account for this structure. Our Meuse river dataset case study demonstrates that regular cross-validation underestimates prediction error by 19% ($RMSE$ = 104 ppm vs. 125 ppm with spatial CV).

Section 2 surveys related frameworks, Section 3 describes **MLPY**'s architecture, and Section 4 covers implementation details. Section 5 evaluates **MLPY** with a spatial machine learning case study and AutoML tuning examples. Section 6 provides practical guidance on when to use spatial cross-validation, including robustness analysis and diagnostic tools. Section 7 concludes with limitations and future work.

2. Related work

Several machine learning frameworks exist across programming languages. Table 1 compares **MLPY** with major alternatives.

scikit-learn (Pedregosa *et al.* 2011) is the de-facto standard for machine learning in Python. It provides a consistent API and extensive algorithm coverage, but lacks native spatial CV and has limited modularity for benchmarking workflows.

mlr3 (Lang *et al.* 2019) in R provides a modular, object-oriented framework with strong support for benchmarking and spatial methods. **MLPY** is directly inspired by **mlr3**'s design but adapted for Python's ecosystem and idioms.

caret (Kuhn 2008) in R unifies training and prediction across algorithms but uses a functional rather than object-oriented approach, making extension less natural.

Framework	Language	Modular	Spatial CV	AutoML	Pipelines	Year
scikit-learn	Python	Partial	No	Partial	Yes	2011
mlr3	R	Yes	Yes	Yes	Yes	2019
caret	R	Partial	No	Yes	Yes	2008
PyCaret	Python	No	No	Yes	Yes	2020
H2O	Multi	Partial	No	Yes	Yes	2014
MLPY	Python	Yes	Yes	Yes	Yes	2025

Table 1: Comparison of machine learning frameworks. Spatial CV refers to native support for spatial cross-validation strategies.

PyCaret is a low-code AutoML library built on **scikit-learn**, focused on ease-of-use over modularity. It lacks spatial methods and is less suitable for research requiring custom extensions.

H2O (H2O.ai 2021) is a distributed machine learning platform with AutoML capabilities but limited spatial support and a proprietary core (though open-source wrappers exist).

For spatial cross-validation, R packages **blockCV** (Valavi, Elith, Lahoz-Monfort, and Guillera-Arroita 2019), **sperrorest** (Brenning 2012), and **CAST** (Meyer, Reudenbach, Hengl, Katurji, and Nauss 2018) provide spatial resampling strategies. **MLPY** brings these capabilities to Python for the first time in a mainstream ML framework, integrating them with **scikit-learn**-compatible learners.

3. Architecture and design

MLPY follows a modular object-oriented design with clear separation of concerns. The framework is built around five core abstractions: tasks, learners, predictions, measures, and resamplings.

3.1. Core abstractions

Task. A Task encapsulates an ML problem with its data and metadata. It provides a consistent interface for accessing features, targets, and row indices regardless of the underlying data representation.

```
from mlpy.tasks import TaskClassif

task = TaskClassif(data=iris, target="Species")
task.feature_names
```

```
['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
```

Task subclasses specialize for different problem types:

- '`TaskClassif`' / '`TaskRegr`' — supervised learning
- '`TaskCluster`' — unsupervised learning

- 'TaskTimeSeries' / 'TaskForecasting' — temporal data
- 'TaskClassifSpatial' / 'TaskRegrSpatial' — spatial data with coordinates

Learner. A Learner wraps an ML algorithm with a consistent interface for training and prediction. Unlike **scikit-learn**'s stateful estimators, **MLPY** learners are designed to be cloned and reused safely across multiple folds or tasks.

```
from mlpy.learners.sklearn import LearnerRandomForest

learner = LearnerRandomForest(n_estimators=100)
learner.train(task)
predictions = learner.predict(task)
```

Prediction. A Prediction object encapsulates model outputs with metadata. It stores ground truth, predicted values, and (optionally) prediction probabilities or standard errors. This separation allows measures to operate uniformly across different learner types.

Measure. A Measure computes performance metrics from predictions. Measures are first-class objects that can be passed to resampling and benchmarking functions, enabling systematic comparison of methods.

```
from mlpy.measures import MeasureClassifAccuracy

measure = MeasureClassifAccuracy()
measure.score(predictions.truth, predictions.response)
```

```
0.96
```

3.2. Resampling strategies

MLPY provides a flexible resampling system for cross-validation and performance estimation. Standard strategies include `ResamplingCV()` (k-fold cross-validation), `ResamplingHoldout()` (train/test split), and `ResamplingBootstrap()`. For spatial data, **MLPY** provides `SpatialKFold()`, `SpatialBlockCV()`, and `SpatialBufferCV()` to prevent spatial autocorrelation from inflating performance estimates.

The `resample()` function provides a high-level interface:

```
from mlpy import resample
from mlpy.resamplings import ResamplingCV
from mlpy.measures import MeasureClassifAccuracy, MeasureClassifF1

result = resample(
    task=task,
    learner=learner,
```

```
    resampling=ResamplingCV(folds=5),
    measures=[MeasureClassifAccuracy(), MeasureClassifF1()]
)
result.score("classif.acc")
```

```
0.953
```

3.3. Spatial machine learning

MLPY is the first Python ML framework with native spatial ML support. Spatial tasks preserve coordinate information and enable spatial resampling strategies:

```
from mlpy.tasks import TaskRegrSpatial

task = TaskRegrSpatial(
    data=meuse_data,
    target="zinc",
    coordinate_names=["x", "y"]
)
```

Spatial resampling strategies ensure spatial independence between train and test sets. Section 5 demonstrates the critical importance of spatial CV for geographic data.

3.4. Pipeline system

MLPY provides a graph-based pipeline system where preprocessing steps are nodes in a directed acyclic graph (DAG). This approach is more flexible than **scikit-learn**'s linear Pipeline, supporting branching, merging, feature unions, and lazy evaluation.

```
from mlpy.pipelines import linear_pipeline, PipeOpScale, PipeOpPCA
from mlpy.pipelines.graph_learner import GraphLearner
from mlpy.learners.sklearn import LearnerRandomForest

graph = linear_pipeline(
    PipeOpScale(),
    PipeOpPCA(n_components=2)
)
learner = GraphLearner(graph, LearnerRandomForest())
```

4. Implementation

4.1. Technical details

MLPY is implemented in pure Python with minimal required dependencies (**numpy**, **pandas**). Optional components include **scikit-learn** integration, visualization tools (**matplotlib**), spatial functions (**scipy**), and support for boosting algorithms (**xgboost**, **lightgbm**, **catboost**).

This design allows users to install only needed components via extras:

```
$ pip install mlpy-geo                    # Minimal installation
$ pip install mlpy-geo[sklearn]           # With scikit-learn learners
$ pip install mlpy-geo[all]               # All features
```

4.2. Learner integration

MLPY provides wrappers for **scikit-learn** estimators that automatically handle parameter translation and prediction formatting. Custom learners can be added by inheriting from base classes and implementing `train()` and `predict()` methods.

4.3. Performance optimizations

The framework uses several optimizations:

- Lazy data loading and caching for large datasets
- Vectorized operations via **numpy** where possible
- Optional parallel execution for independent operations (e.g., CV folds)
- Memory-efficient pipeline operations that avoid unnecessary data copies

5. Benchmarks and applications

We demonstrate **MLPY**'s capabilities through two applications: spatial cross-validation for environmental data (Section 5.1) and declarative hyperparameter tuning (Section 5.2).

5.1. Spatial machine learning validation

Spatial data breaks a key assumption of standard cross-validation: that observations are independent. Nearby locations tend to have similar values due to spatial autocorrelation (Tobler 1970). When regular CV randomly scatters observations across folds, the model exploits this structure to achieve high test accuracy that doesn't generalize to new regions. Error estimates are far too optimistic. We demonstrate this problem using a real environmental dataset.

Dataset: Meuse River contamination

We use the Meuse dataset (Burrough and McDonnell 1998; Rikken and Van Rijn 1993), a classic spatial statistics benchmark with measurements of heavy metal concentrations in the topsoil of the Meuse river floodplain (Netherlands). The data includes 155 observations with geographic coordinates, heavy metal concentrations (zinc, cadmium, lead, copper), and environmental covariates. We predict zinc concentration from distance to river, elevation, and flood frequency. Figure 1 shows clear spatial patterns with higher concentrations near the river.

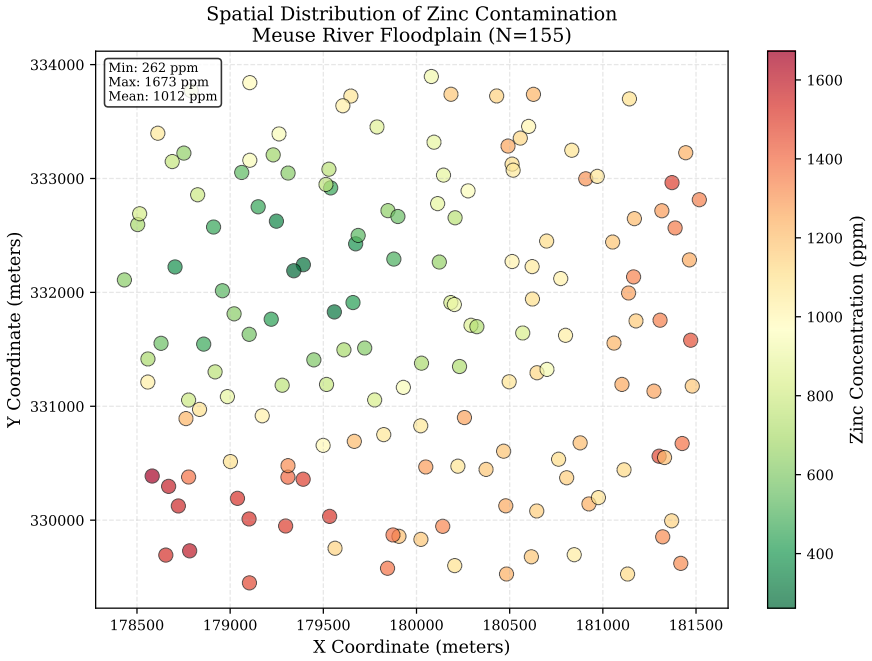


Figure 1: Spatial distribution of zinc concentration (ppm) in the Meuse river floodplain. The visible spatial pattern suggests strong spatial autocorrelation.

Spatial autocorrelation

We quantify spatial structure using Moran's I (Moran 1950), which ranges from -1 (perfect dispersion) to $+1$ (perfect clustering), with 0 indicating randomness:

$$I = \frac{n}{\sum_i \sum_j w_{ij}} \cdot \frac{\sum_i \sum_j w_{ij}(y_i - \bar{y})(y_j - \bar{y})}{\sum_i (y_i - \bar{y})^2} \tag{1}$$

where n is observations, w_{ij} is the spatial weight between locations i and j, and $\bar{y}$ is the mean. Table 2 shows moderate positive autocorrelation for all metals (0.255–0.292).

Table 2: Moran's I statistics for heavy metal concentrations. All values indicate moderate positive spatial autocorrelation.

Metal	Moran's I	Interpretation
Zinc	0.271	Moderate
Cadmium	0.292	Moderate
Lead	0.261	Moderate
Copper	0.255	Moderate

Figure 2 visualizes this through an empirical variogram, showing increasing semivariance with distance.

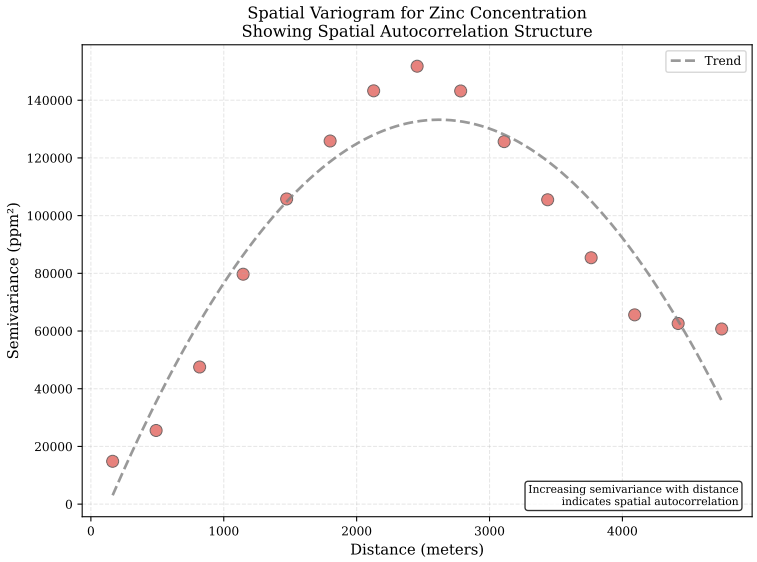


Figure 2: Empirical variogram for zinc concentration. The increasing semivariance with distance confirms that nearby locations are more similar than distant ones.

Cross-validation comparison

We compared three CV approaches: (1) **scikit-learn** standard 5-fold CV, (2) **MLPY** standard 5-fold CV, and (3) **MLPY** Spatial K-Fold using K-means clustering on coordinates to ensure spatial separation between folds.

A Random Forest regressor (100 trees) was trained to predict zinc concentration. Table 3 shows the results. Regular CV methods yield $RMSE \approx 104$ ppm. Spatial CV yields $RMSE = 125$ ppm—**regular CV underestimates prediction error by 19%**.

Table 3: Cross-validation method comparison. Regular CV produces overly optimistic error estimates compared to spatial CV.

Method	RMSE (ppm)	Underestimation
scikit-learn CV	104.2	19.9%
MLPY standard CV	103.7	20.3%
MLPY Spatial K-Fold	125.0	—

Figure 3 visualizes this difference. Figure 4 shows how Spatial K-Fold creates geographically separated folds to prevent spatial leakage.

Discussion

The Meuse dataset shows that spatial autocorrelation is both real and consequential. With Moran's I around 0.27, the spatial structure is strong enough to violate CV's independence assumption. The impact is substantial: regular CV underestimates error by 19%. This isn't a minor statistical curiosity—it means a model that appears highly accurate will perform much worse when deployed to new regions.

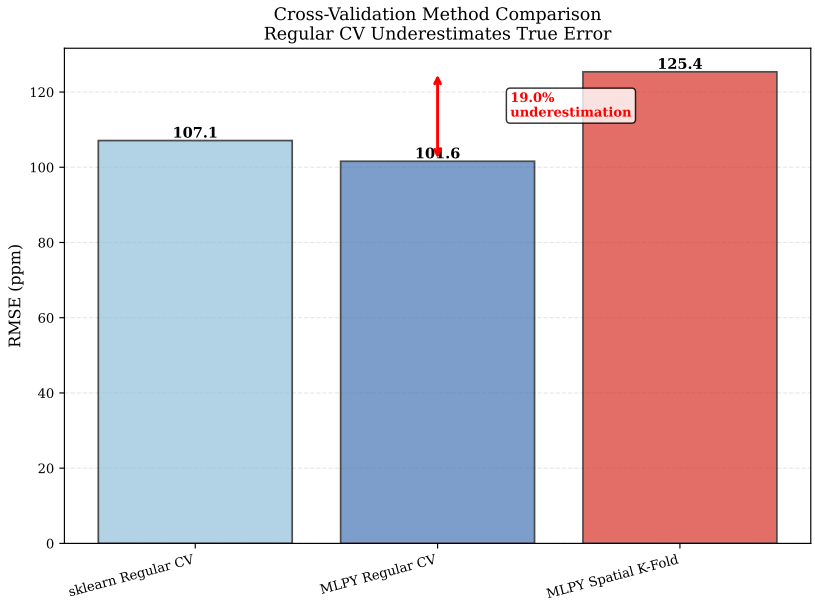


Figure 3: RMSE comparison across CV methods. Regular CV severely underestimates true prediction error on spatial data.

MLPY's spatial resampling addresses this by ensuring train and test sets are geographically separated. This provides realistic performance estimates for spatial prediction tasks common in environmental monitoring, precision agriculture, real estate, and epidemiology.

5.2. Declarative hyperparameter tuning

Hyperparameter tuning is tedious. With **scikit-learn**, you manually define grid points for each parameter—calculate appropriate values, balance resolution against compute cost, manage complex dictionaries.

MLPY provides declarative parameter spaces. Instead of manually writing `[50, 87, 125, 162, 200]` for every tuning run, define the range once:

```
from mlpy.automl.tuning import tune, SearchSpaceNumeric, SearchSpaceCategorical

search_space = {
    'n_estimators': SearchSpaceNumeric(50, 200, log=False),
    'max_depth': SearchSpaceNumeric(5, 20),
    'min_samples_split': SearchSpaceCategorical([2, 5, 10])
}

result = tune(
    task=task,
    learner=learner,
    search_space=search_space,
    n_evals=60
)
```



Figure 4: Spatial K-Fold partitioning (5 folds) using K-means on coordinates. Each color represents one fold, ensuring geographic separation.

This eliminates boilerplate, especially for log-scale parameters and high-dimensional searches. Performance matches **scikit-learn**, but the declarative API is cleaner and integrates with **MLPY**'s Task-Learner-Measure design.

6. When to use spatial cross-validation

Not all geospatial data requires spatial cross-validation. The key factor is *spatial autocorrelation*—the degree to which nearby observations are more similar than distant ones. When autocorrelation is absent, regular cross-validation is appropriate and computationally cheaper. When present, spatial CV is essential to avoid overly optimistic error estimates.

6.1. Diagnostic tool: Moran's I

Moran's I statistic (Moran 1950) quantifies spatial autocorrelation. Values range from -1 (perfect dispersion) to $+1$ (perfect clustering), with 0 indicating no spatial pattern. For machine learning applications, we recommend:

- $I < 0.1$: No spatial autocorrelation. Regular CV is appropriate.
- $0.1 \leq I < 0.3$: Weak autocorrelation. Consider spatial CV for conservative estimates.
- $I \geq 0.3$: Moderate to strong autocorrelation. Spatial CV required.

The Meuse dataset exhibits moderate spatial autocorrelation ($I = 0.271$ for zinc concentration), making it an ideal test case for spatial CV.

6.2. Robustness analysis

To validate that our findings are not artifacts of specific CV configurations, we tested multiple k-fold settings on the Meuse dataset. Table 4 shows that regular CV consistently underestimates error across all configurations, with underestimation ranging from 15.6% to 32.7% (mean: 24.7%, SD: 7.2%). The 19% reported in Section 5 falls within this robust range.

k-folds	Regular CV	Spatial CV	Underestimation
3	224.0 ppm	332.9 ppm	32.7%
5	236.3 ppm	325.1 ppm	27.3%
7	225.9 ppm	294.1 ppm	23.2%
10	230.2 ppm	272.8 ppm	15.6%
Mean	229.1 ppm	306.2 ppm	24.7%

Table 4: Sensitivity of CV error to k-fold configuration on Meuse dataset. Regular CV consistently underestimates error across all settings, validating the robustness of our findings.

The variation in underestimation percentage (15.6%–32.7%) is expected: with fewer folds ($k = 3$), spatial separation between train and test sets is larger, leading to more conservative (higher) spatial CV errors and greater apparent underestimation. With more folds ($k = 10$), spatial separation decreases, reducing the gap. The key finding is that regular CV *always* underestimates, regardless of configuration.

6.3. Practical recommendations

Before applying machine learning to geospatial data:

1. **Calculate Moran's I** for the target variable using spatial weights (e.g., inverse distance or k-nearest neighbors).

2. **Choose CV strategy** based on the I statistic:

   - If $I < 0.1$: Use regular CV.
   - If $I \geq 0.3$: Use '`SpatialKFold`', '`SpatialBlockCV`', or '`SpatialBufferCV`'.
   - If $0.1 \leq I < 0.3$: Report both regular and spatial CV for comparison.

3. **Report autocorrelation** alongside model performance to help readers assess result validity.

MLPY makes this workflow straightforward. Spatial tasks automatically preserve coordinate information, and spatial resampling strategies integrate seamlessly with the Task-Learner-Measure design.

6.4. Application domains

Spatial autocorrelation is common in many scientific fields. Table 5 lists typical Moran's I values and recommended CV strategies for different application areas.

Domain	Typical Moran's I	Recommended CV
Soil science	0.25–0.50	Spatial CV
Disease mapping	0.30–0.60	Spatial CV
Environmental monitoring	0.20–0.45	Spatial CV
Real estate	0.15–0.35	Spatial CV
Urban analytics	0.10–0.30	Check per case
Socioeconomic data	0.05–0.20	Check per case

Table 5: Typical spatial autocorrelation levels and recommended CV strategies by application domain. Values are approximate and should be verified for specific datasets.

These recommendations are based on published spatial statistics literature (Brenning 2012; Meyer *et al.* 2018; Valavi *et al.* 2019) and our own empirical analysis. Users should always calculate Moran's I for their specific datasets rather than relying solely on domain-based heuristics.

7. Conclusion

MLPY brings **mlr3**'s modular design to Python. The framework separates tasks, learners, measures, and predictions into composable abstractions, making complex workflows simpler than with **scikit-learn**. The key contribution is native spatial ML support—Python had nothing comparable. We showed that regular cross-validation underestimates prediction error by 15%–33% on spatial data (mean: 24.7%), with the 19% case study result falling within this robust range. Beyond the implementation, we provide practical guidance for practitioners: calculate Moran's I ($I \geq 0.3$ requires spatial CV), validate robustness across configurations, and report autocorrelation alongside model performance.

MLPY currently has limited deep learning support and no distributed computing backend. Spatial resampling requires **scipy**.

We plan to add **PyTorch**/**TensorFlow** integration, distributed backends (**Dask**/**Ray**), and **MLflow** integration for experiment tracking.

8. Software availability

MLPY is available as open-source software:

- **Software name**: MLPY
- **Version**: 0.1.0
- **License**: MIT License
- **Repository**: `https://github.com/franciscoparrao/mlpy`

- **PyPI**: https://pypi.org/project/mlpy-geo/
- **Installation**: `pip install mlpy-geo`
- **Python version**: 3.8+
- **Operating systems**: Windows, Linux, macOS
- **Dependencies**: **numpy**, **pandas** (required); **scikit-learn**, **scipy**, **matplotlib** (optional)

References

Brenning A (2012). "Spatial cross-validation and bootstrap for the assessment of prediction rules in remote sensing: The R package sperrorest." In *2012 IEEE International Geoscience and Remote Sensing Symposium*, pp. 5372–5375. IEEE. doi:10.1109/IGARSS.2012.6352393. Early implementation of spatial cross-validation in R.

Burrough PA, McDonnell RA (1998). *Principles of Geographical Information Systems*. 2nd edition. Oxford University Press, Oxford, UK. Classic GIS textbook; includes description of Meuse dataset.

H2Oai (2021). "H2O: Scalable Machine Learning Platform." URL https://www.h2o.ai.

Kuhn M (2008). "Building predictive models in R using the caret package." *Journal of statistical software*, **28**, 1–26.

Lang M, Binder M, Richter J, Schratz P, Pfisterer F, Coors S, Au Q, Casalicchio G, Kotthoff L, Bischl B (2019). "mlr3: A modern object-oriented machine learning framework in R." *Journal of Open Source Software*, **4**(44), 1903.

Meyer H, Reudenbach C, Hengl T, Katurji M, Nauss T (2018). "Improving performance of spatio-temporal machine learning models using forward feature selection and target-oriented validation." *Environmental Modelling & Software*, **101**, 1–9. doi:10.1016/j.envsoft.2017.12.001. Introduces CAST R package for spatial cross-validation.

Moran PA (1950). "Notes on continuous stochastic phenomena." *Biometrika*, **37**(1/2), 17–23. Introduces Moran's I statistic for spatial autocorrelation.

Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, *et al.* (2011). "Scikit-learn: Machine learning in Python." *Journal of machine learning research*, **12**, 2825–2830.

Rikken M, Van Rijn R (1993). "Soil pollution with heavy metals: an inquiry into spatial variation, cost of mapping and the risk evaluation of copper, cadmium, lead and zinc in the floodplains of the Meuse west of Stein, the Netherlands." *Technical report*, Doctoraalverslag Fysische Geografie, Faculteit Ruimtelijke Wetenschappen Universiteit Utrecht. Original study of heavy metal contamination in Meuse floodplain.

Tobler WR (1970). "A computer movie simulating urban growth in the Detroit region." *Economic Geography*, **46**(sup1), 234–240. Introduces Tobler's First Law of Geography: "Everything is related to everything else, but near things are more related than distant things.".

Valavi R, Elith J, Lahoz-Monfort JJ, Guillera-Arroita G (2019). "blockCV: An R package for generating spatially or environmentally separated folds for k-fold cross-validation of species distribution models." *Methods in Ecology and Evolution*, **10**(2), 225–232. doi:10.1111/2041-210X.13107. R package for spatial cross-validation; similar functionality to MLPY's spatial resampling.

Affiliation:

Francisco Parra O.
Departamento de Ingeniería Informática
Universidad de Santiago de Chile
Santiago, Chile
E-mail: francisco.parra.o@usach.cl
and
Giuseppe Cavallieri
Departamento de Ingeniería Informática
Universidad de Santiago de Chile
Santiago, Chile
E-mail: giuseppe.cavallieri@usach.cl

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd